Adversarial Co-Training for Agentic Coding

Team Members: Daniel Choi, Jack Li, Muyu Pan, Tianyou Zhao

PROBLEM & MOTIVATION

- Reinforcement Learning (RL) for code generation requires unit test feedback
- Unit tests must be high quality and diverse to provide valid learning signals and prevent reward hacking
- Existing implementations have limited exposure to runtime state of code
- Adversarial co-training drives continuous mutual improvement between code generation and bug discovery (similar to GANs)

ADVERSARIAL GENERATOR–TESTER LOOP

USER REQUEST
Problem specification + function signature

›

CODE LLM
Generate code

›

TEST LLM
Find adversarial scenarios

›

VERIFIER
Run tests on code + extract rewards + update model weights

HYPOTHESIS: Providing branch coverage and selected runtime values to the test LLM during adversarial co-training will help it generate improved valid unit tests that detect more bugs under a fixed generation budget, leading to improved coder performance.

EXPERIMENT SETUP (based on Coder-A1 paper):

Model: Qwen2.5-Coder-Instruct (1.5B)

Training Dataset: KodCode-V1

Tester Eval: UnLeakedTestBench

Coder Eval: HumanEval + BigCodeBench

Tools: verl (RL framework), vLLM (speed up inference), Sandbox Fusion (sandboxed environments)

TIMELINE & MAJOR MILESTONES (7 WEEKS)

WEEKS 1–2

Prepare data & baselines

- Set up KodCodeV1 & benchmarks
- Integrate verl & vLLM with Qwen 1.5B

WEEK 3

Replicate Code-A1

- Implement & test Code-A1 pipeline
- Verify baseline replication

WEEKS 4–5

Implement improvements

- Add execution feedback: branch coverage & runtime values

WEEKS 6–7

Testing & report writing

- Benchmark eval
- Final project report

EXPECTED OUTCOME & TARGET IMPACT

- Evaluate whether richer training signals (branch coverage, select runtime values) for the Test LLM produce higher quality tests.
- Show that enhanced adversarial co-training improves coder and tester performance across HumanEval, BigCodeBench, and UnLeakedTestBench.

Adversarial Co-Training for Agentic Coding

Team Members: Daniel Choi, Jack Li, Muyu Pan, Tianyou Zhao

PROBLEM & MOTIVATION

CORE PROBLEM INVESTIGATED

- Reinforcement Learning (RL) for code generation requires unit test feedback
- Unit tests must be high quality and diverse to provide valid learning signals and prevent reward hacking
- Existing implementations have limited exposure to runtime state of code

KEY MOTIVATION

- Adversarial co-training drives continuous mutual improvement between code generation and bug discovery (similar to GANs).

CO-TRAINING ARCHITECTURE: ADVERSARIAL GENERATOR–TESTER LOOP

USER REQUEST

Problem specification + function signature

›

CODE LLM

Generate code

›

TEST LLM

Find adversarial scenarios

›

VERIFIER

Run tests on code + extract rewards + update model weights

MINIMAX OBJECTIVE

$$\min_{\theta} \max_{\phi} \mathbb{E}_{p,C,\tau}[V(p, C, \tau)]$$

V: verified bug severity score

C: generated code solution

τ: adversarial test scenario

p: programming problem

HYPOTHESIS: Providing branch coverage and selected runtime values to the test LLM during adversarial co-training will help it generate improved valid unit tests that detect more bugs under a fixed generation budget, leading to improved coder performance.

Georgia Tech

Project Methodology, Timeline & Outcomes

APPROACH & EXPERIMENTAL SETUP

MODEL & RL FRAMEWORKS

- **Base Model:** Qwen2.5-Coder-Instruct (1.5B)
- **verl:** Coordinates generation, reward calculation & RL updates
- **vLLM:** High-throughput LLM response generation engine

TRAINING & TEST BENCHES

- **Training:** KodCode-V1
- **Tester Eval:** UnLeakedTestBench
- **Coder Eval:** HumanEval + BigCodeBench

EXPERIMENTS

- **Baseline Replication:**
  Reproduce Code-A1 co-training with verl & vLLM
- **Modify Training Signal:**
  Feed branch coverage & select runtime values to Test LLM

TIMELINE & MAJOR MILESTONES (7 WEEKS)

WEEKS 1–2

Prepare data & baselines

- Set up KodCodeV1 & benchmarks
- Integrate verl & vLLM with Qwen 1.5B

WEEK 3

Replicate Code-A1

- Implement & test Code-A1 pipeline
- Verify baseline replication

WEEKS 4–5

Implement improvements

- Add execution feedback: branch coverage & runtime values

WEEKS 6–7

Testing & report writing

- Benchmark eval (HumanEval, MBPP, etc.)
- Ablations & final project report

EXPECTED OUTCOME & TARGET IMPACT

- Evaluate whether richer training signals (branch coverage, select runtime values) for the Test LLM produce higher quality tests.
- Show that enhanced adversarial co-training improves coder and tester performance across HumanEval, BigCodeBench, and UnLeakedTestBench.

Georgia Tech